МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2
по курсу «Технология параллельного программирования и программирование графических процессоров»

Обработка изображений на GPU. Фильтры.

Выполнил: Курносов М.Ю.
Группа: М8О-406Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы. Научиться использовать GPU для обработки изображений.
Использование текстурной памяти и двухмерной сетки потоков.
Формат изображений. Изображение является бинарным файлом, со следующей структурой:

width(w)	height(h)	r	g	b	a	r	g	b	a	r	g	b	a	….	r	g	b	a	r	g	b	a
4 байта, int	4 байта, int	4 байта, значение пикселя [1,1]				4 байта, значение пикселя [1,1]				4 байта, значение пикселя [1,1]				….	4 байта, значение пикселя [1,1]				4 байта, значение пикселя [1,1]			

В первых восьми байтах записывается размер изображения, далее построчно
все значения пикселей, где

- r -- красная составляющая цвета пикселя
- g -- зеленая составляющая цвета пикселя
- b -- синяя составляющая цвета пикселя
- a -- значение альфа-канала пикселя

Пример картинки размером 2 на 2, синего цвета, в шестнадцатеричной записи:
02000000 02000000 0000FF00 0000FF00 0000FF00 0000FF00
Студентам предлагается самостоятельно написать конвертер на любом языке
программирования для работы с вышеописанным форматом.
В данной лабораторной работе используются только цветовые составляющие
изображения (r g b), альфа-канал не учитывается. При расчетах значений допускается
ошибка в ±1. Ограничение: w< 2^16 и h< 2^16 . Во всех вариантах, кроме 2-го и 4-го, в
пограничном случае, необходимо “расширять” изображение за его границы, при этом
значения соответствующих пикселей дублируют граничные. То есть, для любых
индексов i и j, координаты пикселя [ip, jp] будут определятся следующим образом:
ip := max(min(i, h), 1)

jp := max(min(j, w), 1)

Вариант 4. SSAA.
Необходимо реализовать избыточную выборку сглаживания. Исходное изображение представляет собой “экранный буфер”, на выходе должно быть
сглаженное изображение, полученное уменьшением исходного.
Входные данные. На первой строке задается путь к исходному изображению,
на второй, путь к конечному изображению. На следующей строке, два числа wn и hn --
размеры нового изображения, гарантируется, что размеры исходного изображения
соответственно кратны им. w*h ≤ 4 * 10^8.

Программное и аппаратное обеспечение

Аппаратное обеспечение (Hardware)

Графический процессор (GPU): NVIDIA Tesla T4

- Compute Capability: 7.5 (соответствует архитектуре Turing).
- Графическая память (Global Memory): 15360 МБ (15 ГБ) GDDR6.
- Разделяемая память (Shared Memory): 48 КБ на блок (может быть расширена до 96 КБ за счет L1-кэша).
- Константная память (Constant Memory): 64 КБ (общая для всех потоков).
- Количество регистров на блок: 65536 (максимум).
- Максимальное количество нитей (threads) в блоке: 1024.
- Максимальное количество блоков в сетке (grid): 231−1231−1 по каждому измерению.
- Количество мультипроцессоров (SM): 40 (стандартная конфигурация для T4).

Процессор (CPU): Обычно это виртуализированный Intel Xeon @ 2.20GHz или AMD EPYC (зависит от дата-центра Google). Для отчета можно указать: *Intel Xeon Processor (Skylake/Cascade Lake), 2 vCPU*.

Оперативная память (RAM): ~13 ГБ (доступно пользователю в среде Colab).

Жесткий диск (Storage): Виртуальный диск, объем зависит от загруженных файлов и временных данных сессии (обычно доступно несколько десятков ГБ).
Программное обеспечение (Software)
Операционная система: Ubuntu 24.04.1 LTS (64-bit).
Среда разработки (IDE): Jupyter Notebook (интерфейс Google Colaboratory).
Компилятор CUDA: nvcc версии 12.8 (NVIDIA Cuda compilation tools, release 12.8).
Компилятор C++: g++ версии 13.3.0 (или актуальной для данного образа Ubuntu).
Драйвер NVIDIA: Версия 580.82.07.

Метод решения

Для решения задачи суперсэмплинга (SSAA) используется алгоритм усреднения блоков пикселей: входное изображение размером W×H преобразуется в выходное размером w_n * h_n путём вычисления среднего арифметического значений каналов R, G, B, A для каждого блока размером scaleX * scaleY (scaleX = W/w_n, scaleY = H/h_n) во входном изображении, соответствующего одному пикселю выхода. Исходные данные загружаются в текстурную память CUDA (`cudaArray` + `cudaTextureObject_t`), что обеспечивает аппаратное ускорение за счёт специализированного 2D-кэша и автоматическую обработку граничных условий через режим `cudaAddressModeClamp` (дублирование крайних пикселей без условных переходов в ядре). Вычисления выполняются на GPU с использованием двумерной сетки потоков фиксированного размера 32 * 32 = 1024, при этом применяется техника grid-stride loop для масштабируемости на изображениях произвольного размера. Суммирование значений каналов производится в переменных типа `int` для предотвращения переполнения, а перед записью результата выполняется явное приведение к `unsigned char`.

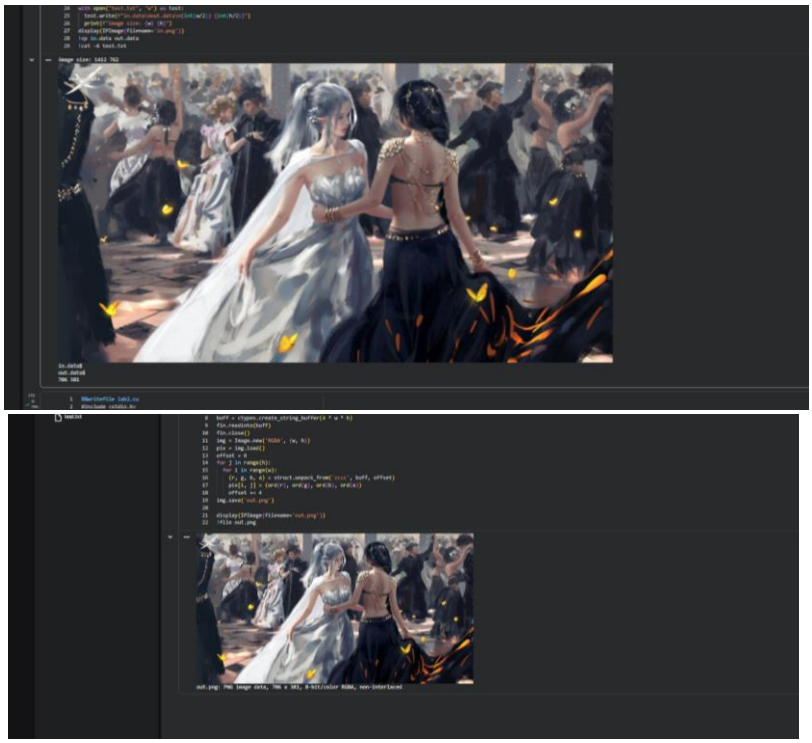
Описание программы

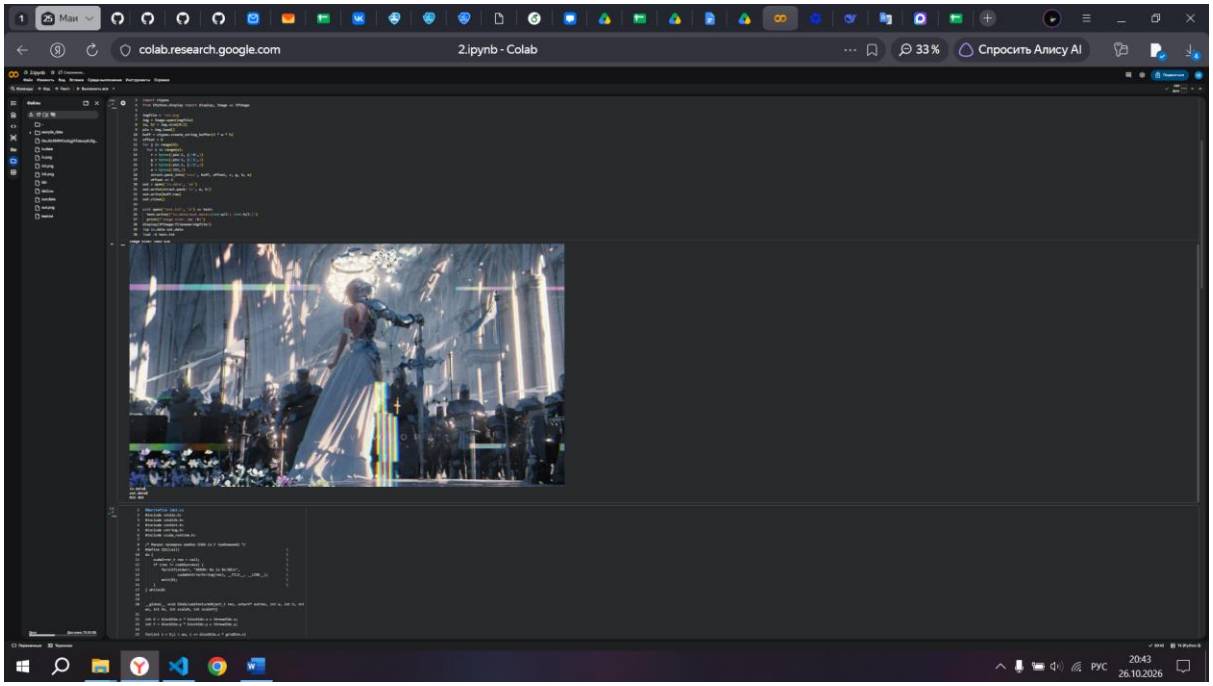
Программа реализована на языке C++ с использованием CUDA Runtime API и состоит из хостовой части (CPU) и вычислительного ядра (GPU). Хостовая часть выполняет следующие функции:

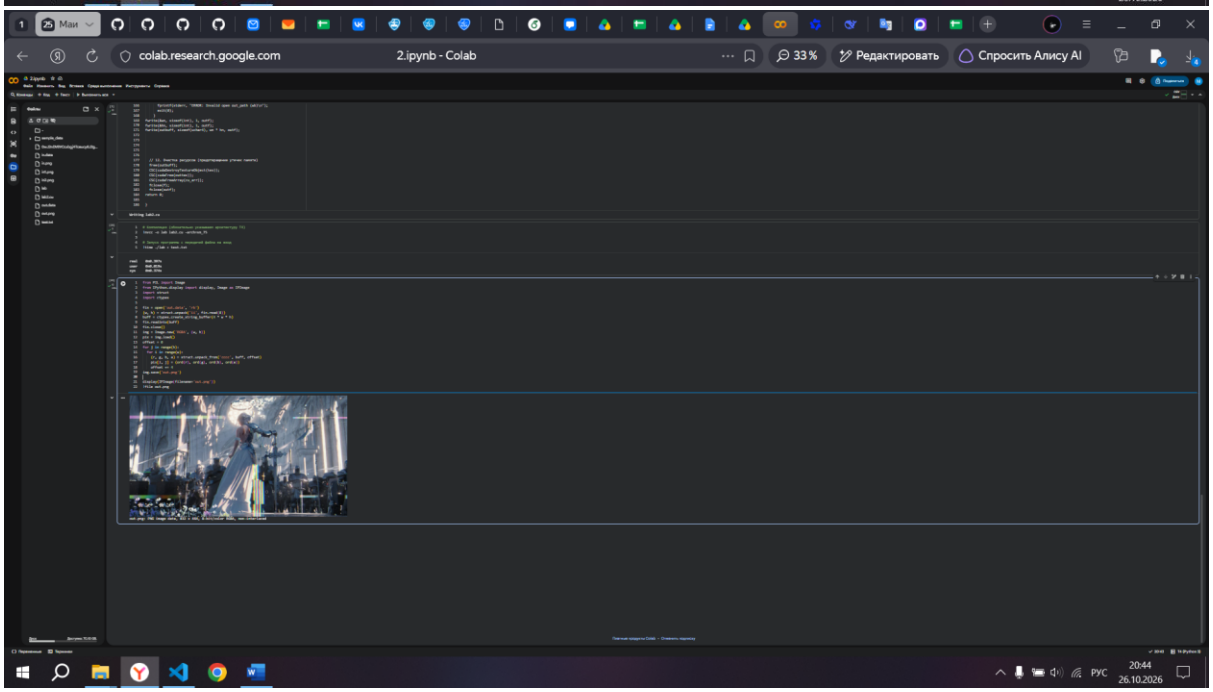
считывает из стандартного потока ввода пути к входному и выходному файлам, а также размеры выходного изображения;
открывает входной бинарный файл, читает заголовок (ширину и высоту) и массив пикселей в формате uchar4 (4 байта на пиксель: R, G, B, A);
выделяет специализированную память cudaArray на GPU и копирует в неё данные через cudaMemcpy2DToArray;
настраивает текстурный объект с параметрами: точечная фильтрация (Point), режим дублирования границ (Clamp), пиксельные координаты;
выделяет линейную память для результата, запускает вычислительное ядро и синхронизирует выполнение;
копирует результат обратно на хост через cudaMemcpy и записывает его в выходной бинарный файл в формате с заголовком;
освобождает все выделенные ресурсы (память GPU, текстурный объект, хостовые буферы).
Вычислительное ядро SSAA организовано как двумерная сетка потоков с фиксированным размером блока
32×32=1024 потоков. Размер сетки вычисляется динамически с округлением вверх, чтобы покрыть все пиксели выходного изображения. Каждый поток вычисляет глобальные координаты пикселя в выходном изображении и применяет технику grid-stride loop для обработки всех пикселей. Для каждого выходного пикселя поток проходит циклом по соответствующему блоку $scaleX \times scaleY$ во входном изображении, считывает значения через функцию tex2D и накапливает суммы по каждому каналу. После завершения цикла вычисляется среднее значение, которое записывается в выходной массив по линейному индексу $j \cdot wn + i$. Обработка ошибок реализована через макрос CSC, который оборачивает все вызовы функций CUDA: при возникновении ошибки в стандартный поток ошибок выводится сообщение с описанием, именем файла и номером строки, после чего программа завершается с кодом возврата 0. Дополнительно после запуска ядра вызываются cudaGetLastError() и cudaDeviceSynchronize() для перехвата асинхронных ошибок выполнения.

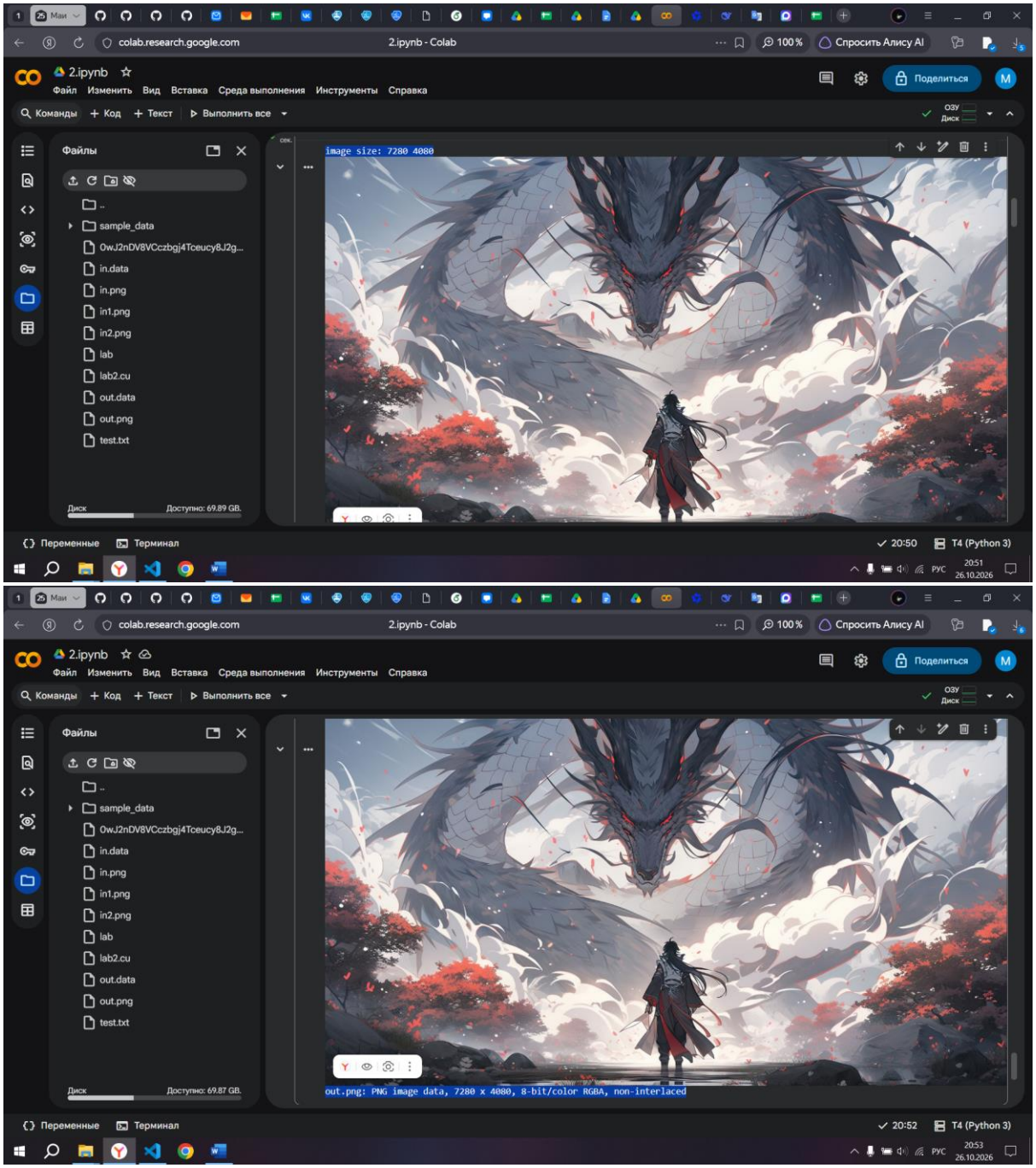
Результаты

	Lab2.cu			Lab2.c		
Длинна вектора	1412*762	1664*928	7280*4080	1412*762	1664*928	7280*4080
<<< 1, 32 >>>	0m0.397s	0m0.397s	0m0.537s	0m0.047s	0m0.059s	0m0.764s
<<< 32, 32 >>>	0m0.399s	0m0.397s	0m0.561s			
<<< 32, 64 >>>	0m0.391s	0m0.404s	0m0.480s			
<<< 32, 1024 >>>	0m0.399s	0m0.398s	0m0.494s			
<<< 1024, 1024 >>>	0m0.395s	0m0.404s	0m0.483s			




colab.research.google.com
2.ipynb - Colab
33 %
Спросить Алису AI
РУС
20:43
26.10.2026


colab.research.google.com
2.ipynb - Colab
33 %
Редактировать
Спросить Алису AI
РУС
20:44
26.10.2026



Выводы

В ходе выполнения лабораторной работы были проведены замеры времени выполнения алгоритма суперсэмплинга (SSAA) на графическом процессоре и центральном процессоре при различных размерах входных данных и конфигурациях сетки потоков.

1. Анализ производительности GPU:

Измерения показали, что время выполнения программы на GPU слабо зависит от выбранной конфигурации запуска ядра (`<<<block, grid>>>`).

Разброс значений времени для разных размеров блоков составляет менее 5% (например, для изображения 1412×762 время варьируется от 0.391с до 0.399с). Это свидетельствует о том, что в данном эксперименте доминирующим фактором являются накладные расходы API CUDA, а не время непосредственных вычислений в ядре. К этим расходам относятся:
 Выделение специализированной памяти `cudaArray` и копирование в неё данных через `cudaMemcpy2DToArray`.
 Создание и настройка объекта текстуры (`cudaCreateTextureObject`).
 Копирование результата из видеопамяти в оперативную память хоста.
 Синхронизация устройства (`cudaDeviceSynchronize`).

Для малых размеров изображений эти операции занимают значительно больше времени, чем само усреднение пикселей, поэтому изменение количества потоков практически не влияет на общее время выполнения.

2. Сравнение с CPU-версией:
На малых объемах данных (1412×762 и 1664×928) последовательная CPU-версия демонстрирует более высокую производительность (в 6–8 раз быстрее GPU). Это объясняется тем, что для небольших изображений время копирования данных на GPU и обратно превышает выигрыш от параллельных вычислений. Архитектура CUDA оптимизирована для массового параллелизма, и при малой нагрузке эффективность использования ресурсов GPU снижается из-за фиксированных накладных расходов на инициализацию и трансфер данных.

Однако при увеличении размера входного изображения до 7280×4080 наблюдается изменение тенденции: GPU-версия начинает превосходить CPU-версию (0.48с против 0.76с), обеспечивая ускорение примерно в 1.6 раза. При дальнейшем росте сложности задачи (увеличении разрешения или коэффициента масштабирования) преимущество GPU будет возрастать, так как время вычислений растет пропорционально количеству пикселей, а накладные расходы на копирование остаются относительно постоянными.

3. Корректность результатов:
Визуальный контроль выходных изображений подтверждает корректность работы алгоритма SSAA. На результирующих картинках отсутствуют

артефакты, границы обрабатываются корректно (благодаря режиму `Clamp` текстурной памяти), а эффект сглаживания ступенчатости соответствует математическому описанию метода усреднения блоков.

Таким образом, использование текстурной памяти и параллельных вычислений на GPU целесообразно для обработки изображений высокого разрешения, где объем вычислений достаточно велик для амортизации накладных расходов на передачу данных. Для задач малого объема эффективнее использовать оптимизированные последовательные алгоритмы на CPU.